

Tema 1.

Fundamentos de Python para PLN

Nuevo encargo en LinguaAdapt: medir vocabulario, patrones y legibilidad

Prácticas | Programación para PLN | Universidad Pontificia Comillas

Proyecto de práctica

Encargo

LinguaAdapt ha presentado a un cliente institucional un primer prototipo para observar la legibilidad de textos de lenguaje claro. El prototipo ya cuenta palabras, signos de puntuación, oraciones aproximadas, grupos vocálicos e índices numéricos básicos. Ahora el equipo necesita comprobar que esas operaciones se entienden y pueden modificarse con seguridad.

Estas actividades son la siguiente fase del entrenamiento: no se trata de cambiar el objetivo del proyecto, sino de mirar el código por dentro. Vas a contar palabras clave, clasificar palabras por longitud, escribir una función de transformación, adaptar una expresión regular y recuperar las métricas intermedias que sostienen el índice de legibilidad.

La pregunta profesional de fondo sigue siendo la misma que en el manual teórico: cómo convertir intuiciones lingüísticas sobre claridad, accesibilidad y densidad textual en procedimientos computacionales explícitos, revisables y explicables.

Tu papel

Actúa como estudiante en prácticas dentro de LinguaAdapt. Tu código debe funcionar, pero también debe poder ser leído por otra persona del equipo: una solución útil en PLN combina precisión técnica, criterio lingüístico y documentación clara.

EJERCICIOS DE PRÁCTICA

1. Recuento de frecuencia de palabras clave (uso de bucles)

Conocimientos aplicados: Variables, tokenización (`.split()`), bucles (`for`), estructuras condicionales (`if`).

Instrucciones:

```
texto1: El lenguaje claro es una herramienta fundamental para garantizar la accesibilidad
informativa del lenguaje. En un mundo cada vez más globalizado y diverso;
todos los ciudadanos tienen derecho a comprender los mensajes institucionales sin barreras
lingüísticas. Los ciudadanos exigen transparencia y claridad en la comunicación pública, lo
que fortalece la confianza en las instituciones y promueve una participación ciudadana
activa.
El lenguaje claro no solo beneficia a los ciudadanos, sino que también mejora la eficiencia
administrativa al reducir errores y malentendidos. Además, contribuye a la inclusión social
al facilitar el acceso a la información a personas con diferentes niveles de alfabetización y
habilidades lingüísticas.
En resumen, el lenguaje claro es esencial para una comunicación efectiva y democrática,
promoviendo la igualdad de acceso a la información y fortaleciendo la relación entre las
instituciones y los ciudadanos.
```

- Usa el `texto1` y tokenízalo en una lista de palabras.
- Normaliza todas las palabras a minúsculas usando el método adecuado (`.lower()`), ya sea durante la iteración o previamente, para asegurar un conteo preciso.
- Crea un bucle `for` para recorrer la lista de palabras.
- Dentro del bucle, implementa una condición `if` para contar cuántas veces aparece exactamente la palabra "lenguaje" y cuántas veces aparece la palabra "ciudadanos".

- Imprime el resultado final de ambos contadores.

2. Clasificación condicional de palabras (uso de condicionales y `len()`)

Conocimientos aplicados: `if-elif-else`, `len()` para medir longitud, acceso a listas y variables.

Instrucciones:

1. Toma la lista de palabras (`palabras`) generada a partir del `texto1`.
2. Crea un script que itere sobre los *primeros diez* elementos de esta lista.
3. Utiliza una estructura `if-elif-else` para clasificar e imprimir cada una de estas diez palabras con los siguientes rótulos:
 - "Palabra corta" si tiene 4 o menos caracteres.
 - "Palabra intermedia" si tiene entre 5 y 8 caracteres.
 - "Palabra larga" si tiene más de 8 caracteres.

3. Reversión de la normalización (definición de funciones)

Conocimientos aplicados: Definición de funciones propias (`def`) y procesamiento de cadenas de texto.

Instrucciones:

- Define una función llamada `desnormalizar_mayusculas` que tome un único argumento (una cadena de texto, como una oración).
- Esta función debe convertir la cadena completa a MAYÚSCULAS (lo opuesto a la normalización a minúsculas, por ejemplo) y debe devolver el resultado.
- Llama a la función con una de las oraciones que obtuviste al dividir el `texto1` por el punto e imprime el resultado.

4. Extracción de patrones léxicos simples (adaptación de Regex)

Conocimientos aplicados: Importación y uso del módulo `re` (Expresiones Regulares) y la función `re.findall()`.

No te preocupes por no entender todavía muy bien las expresiones regulares. En las próximas semanas haremos una sesión completa de recapitulación sobre este tema. Para hacer este ejercicio, repasa el punto 6 del cuaderno principal de este tema.

Instrucciones:

- Utiliza el `texto2` proporcionado en el cuaderno.
- En lugar de buscar vocales para estimar sílabas, modifica la expresión regular para buscar y contar el número de palabras que contienen la secuencia de letras "te" (tanto en mayúsculas como en minúsculas).
- Utiliza `re.findall()` con el patrón de RegEx modificado y el indicador `re.IGNORECASE` (similar al ejemplo de sílabas).
- Imprime el número total de coincidencias encontradas.

5. Cálculo inverso del índice de legibilidad (manipulación de variables y aritmética)

Conocimientos aplicados: Uso de variables numéricas y operaciones aritméticas (suma, división).

Instrucciones:

- Copia las variables de conteo final obtenidas para el `texto1` (que son `num_palabras`, `num_oraciones`, `num_silabas`).
- Calcula e imprime las siguientes tres métricas intermedias, que son componentes clave de cualquier índice de legibilidad:
- Longitud Media de Palabra (LMP): Divide el número total de sílabas (`num_silabas`) por el número total de palabras (`num_palabras`).
- Longitud Media de Oración (LMO): Divide el número total de palabras (`num_palabras`) por el número de oraciones (`num_oraciones`).
- Imprime ambos resultados con dos decimales de precisión (`round()`).

Rúbrica de evaluación

Se evalúa que el código funcione y que las decisiones técnicas estén justificadas desde el análisis lingüístico de textos claros y legibles.

Criterio	Excelente	Adecuado	En proceso
Tokenización y normalización	Usa <code>split()</code> y minúsculas de forma coherente para preparar el texto antes del recuento.	Tokeniza y normaliza parcialmente, con algún detalle que puede afectar al conteo.	No prepara el texto de forma suficiente para analizarlo.
Bucles y condicionales	Recorre las palabras y aplica condiciones precisas para contar o clasificar resultados.	Usa bucles y condiciones con algún error menor de lógica o alcance.	La iteración o la condición no resuelven la tarea planteada.
Funciones	Define <code>desnormalizar_mayusculas</code> con parámetro, retorno y llamada de prueba correctos.	La función existe y se aproxima al objetivo, aunque falta algún elemento.	No encapsula correctamente la transformación en una función reutilizable.
Expresiones regulares	Adapta el patrón con <code>re.findall()</code> y <code>re.IGNORECASE</code> para localizar la secuencia solicitada.	Usa <code>re.findall()</code> , pero el patrón o el conteo necesita revisión.	No aplica una expresión regular funcional para la búsqueda propuesta.
Métricas de legibilidad	Calcula LMP y LMO con las variables correctas y presenta resultados redondeados.	Calcula las métricas principales con algún error puntual de variable o formato.	No conecta las variables de conteo con las métricas intermedias.
Claridad profesional	El código es legible, ordenado y permite interpretar los resultados desde el contexto <code>LinguaAdapt</code> .	La solución se entiende, aunque podría mejorar nombres, comentarios o presentación de salidas.	La solución es difícil de seguir o no explica qué significan los resultados.